

VERIFICATION OF AXI TO AHB BRIDGE USING UVM

1. Introduction

Modern System-on-Chip (SoC) designs often integrate multiple IP blocks operating on different AMBA protocols. Since AXI and AHB have different architectures and transaction mechanisms, direct communication between them is not possible. An AXI to AHB Bridge is therefore required to convert AXI transactions into equivalent AHB transactions while maintaining protocol compliance and data integrity.

The objective of this project was to develop a complete UVM-based verification environment to verify the functionality of an AXI to AHB Bridge. The environment was designed to generate protocol transactions, monitor DUT behavior, compare expected and actual results, and validate bridge functionality under different operating conditions.

The verification environment supports reset verification, write transaction verification, read transaction verification, wait-state insertion, response verification, and end-to-end protocol conversion checking.

2. Project Objectives

Objective 1 – Verify Reset Functionality

Ensure that the bridge initializes correctly when reset is asserted and released.

How it was achieved

Dedicated AXI Reset and AHB Reset Agents were developed to generate reset conditions and monitor DUT behavior during initialization.

Objective 2 – Verify AXI Write Transactions

Ensure that AXI write transactions are translated correctly into AHB write transactions.

How it was achieved

Write sequences were developed on the AXI side and corresponding AHB write transactions were monitored and compared through the scoreboard.

Objective 3 – Verify AXI Read Transactions

Ensure that AXI read transactions are translated correctly into AHB read transactions.

How it was achieved

Read sequences were generated and returned data was monitored on both interfaces to verify correctness.

Objective 4 – Verify Address Translation

Ensure that addresses generated by the AXI master are correctly propagated to the AHB interface.

How it was achieved

Address information collected by the AXI Monitor and AHB Monitor was compared inside the scoreboard.

Objective 5 – Verify Data Integrity

Ensure that write and read data remain unchanged during protocol conversion.

How it was achieved

Data captured on the AXI interface was compared against data observed on the AHB interface using scoreboard checks.

Objective 6 – Verify Response Handling

Ensure that AHB responses are correctly interpreted by the bridge.

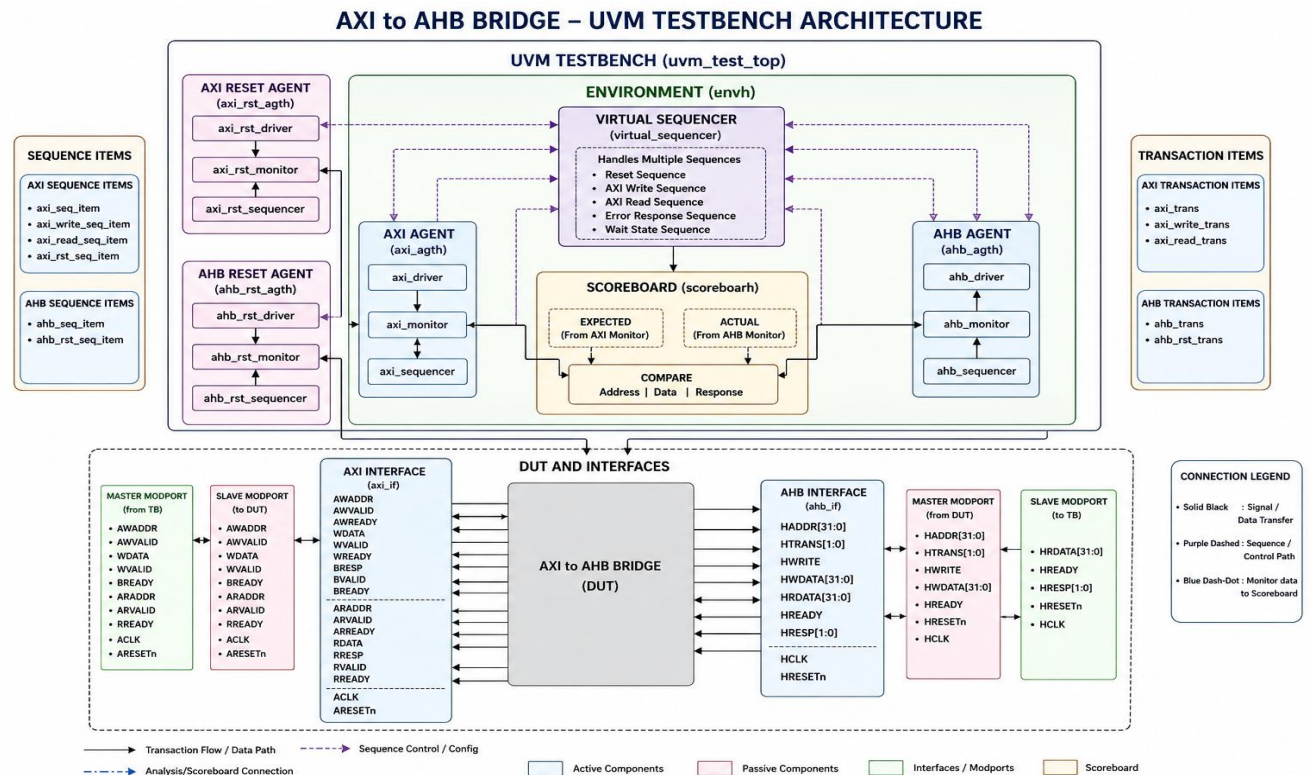
How it was achieved

The AHB slave model was configured to generate OKAY, ERROR, and wait-state conditions and the resulting bridge behavior was verified.

3. DUT Overview

AXI to AHB Bridge

The DUT acts as a protocol converter between an AXI Master and an AHB Slave.



Functions Performed

- AXI Write Transaction Conversion
- AXI Read Transaction Conversion
- Address Translation
- Data Translation
- Response Translation
- Wait-State Handling

→AXI Master

→AXI2AHB Bridge

→AHB Slave

4. Verification Environment

A reusable UVM verification environment was developed to verify the bridge functionality.

The environment consists of:

- AXI Agent
- AHB Agent
- AXI Reset Agent
- AHB Reset Agent
- Virtual Sequencer
- Scoreboard

Verification Flow

AXI Sequence

→ AXI Driver

→ DUT

→ AHB Interface

→ AHB Monitor

→ Scoreboard

5. Master Side Verification (AXI Agent)

The AXI Agent represents the master side of the verification environment.

AXI Driver

The AXI Driver receives sequence items from the sequencer and drives AXI protocol signals onto the DUT interface.

Responsibilities:

- Drive Write Address Channel
- Drive Write Data Channel
- Drive Read Address Channel
- Handle AXI Handshakes

AXI Sequencer

The sequencer controls the flow of transactions and supplies sequence items to the driver.

AXI Monitor

The monitor passively observes protocol activity and reconstructs complete AXI transactions from signal-level behavior.

Captured transactions are forwarded to the scoreboard through analysis ports.

6. Slave Side Verification (AHB Agent)

The AHB Agent models slave-side behavior and generates protocol responses.

AHB Driver

The AHB Driver was designed to generate different response scenarios:

- OKAY Response
- ERROR Response
- Wait-State Conditions

This enabled verification of bridge behavior under normal and abnormal operating conditions.

AHB Sequencer

The sequencer controls the response behavior generated by the driver.

AHB Monitor

The monitor observes AHB transactions and captures:

- Address Information
- Data Information
- Response Information

These transactions are forwarded to the scoreboard.

7. Scoreboard Implementation

The scoreboard is the primary checking component of the verification environment.

It receives transactions from both AXI and AHB monitors and performs end-to-end transaction comparison.

Checks Performed

- Address Comparison
- Data Comparison
- Response Comparison

Scoreboard Flow

AXI Monitor

|

Expected Transaction

AHB Monitor

|

Actual Transaction

↓

Comparison

↓

PASS / FAIL

The scoreboard ensures that every transaction generated on the AXI side is correctly translated and observed on the AHB side.

8. Waveform Analysis

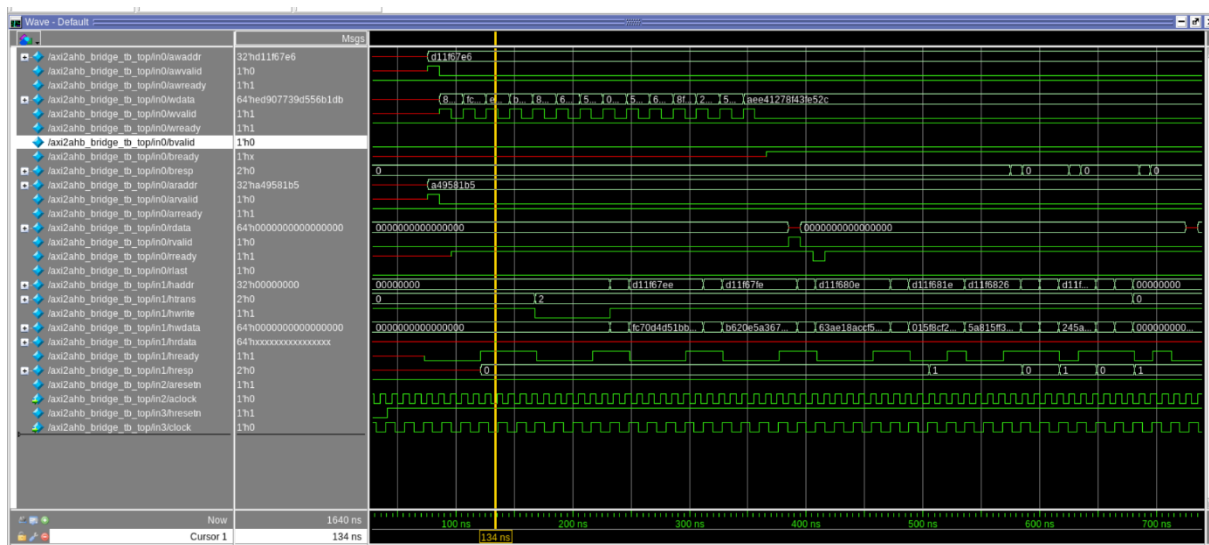
Waveforms were used extensively to validate protocol conversion and debug transaction flow.

Write Transaction Verification

Signals Observed:

- AWVALID
- AWREADY
- WVALID
- WREADY

- The AXI write transaction was successfully accepted by the bridge and translated into an AHB NONSEQ write transfer.



Response Verification

Observation:

HRESP = 00 represented an OKAY response.

HRESP = 01 represented an ERROR response.

The bridge correctly handled both response scenarios.

Reset Verification

Observation:

The DUT initialized correctly after reset assertion and deassertion.

9. Challenges Faced During the Project

Throughout the project several practical challenges were encountered.

Configuration Database Propagation

Passing configuration objects from higher-level components to drivers and monitors required careful debugging of UVM configuration database paths.

Reset Agent Integration

Integrating dedicated AXI and AHB reset agents into the environment required additional synchronization and configuration handling.

Scoreboard Synchronization

Transactions arriving from different protocol interfaces often required timing alignment before comparison.

Protocol Debugging

Debugging protocol handshakes and monitoring transaction flow through multiple layers of the UVM environment required extensive waveform analysis.

UVM Topology Debugging

Ensuring that all agents, sequencers, drivers, monitors, and scoreboard connections were correctly instantiated and connected required multiple iterations.

10. Key Takeaways

This project provided practical exposure to both protocol-level understanding and verification methodology.

Technical Learnings

- AXI Protocol Architecture
- AHB Protocol Architecture
- Protocol Conversion Mechanisms
- UVM Agent Development
- Driver and Monitor Implementation
- Virtual Sequencer Usage
- Scoreboard Design

Verification Learnings

- Transaction-Level Verification
- UVM Factory Mechanism
- Configuration Database Usage
- TLM Communication
- Functional Debugging
- Waveform Analysis

The project significantly improved understanding of both AMBA protocols and industry-standard verification methodologies.

11. Conclusion

A complete UVM-based verification environment was successfully developed for the AXI to AHB Bridge. The environment verified reset functionality, read and write transactions, response handling, and protocol conversion behavior. Through the use of reusable agents, reset agents, a virtual sequencer, monitors, and a scoreboard, the functionality of the bridge was validated under different operating conditions.

The project provided valuable experience in protocol verification, UVM architecture, transaction-level modeling, and debugging techniques commonly used in semiconductor verification projects.